

EVALUACIÓN DE LABORATORIO: SISTEMA DE CLÍNICA CON PUNTEROS GENÉRICOS

RESTRICCIÓN ABSOLUTA: Queda estrictamente prohibido el uso de `struct` o `class`. Toda estructura de datos debe representarse en la memoria dinámica o *heap* utilizando arreglos de punteros genéricos (`void`).

ARCHIVO 2: `funciones.h` y `funciones.cpp`

Debes implementar **exactamente** las siguientes funciones con las firmas descritas:

FASE 1: Funciones de Lectura Específica (Callbacks)

Cada función lee una línea del archivo CSV correspondiente. El delimitador es la coma (,). Si la lectura del primer dato falla (fin de archivo), la función debe retornar `false`.

1. `bool leerPaciente(ifstream& archivo, void*& paciente)`
 - Lee: DNI (int), Nombre (cadena), Edad (int), Género (char).
 - Reserva un arreglo genérico de tamaño `pacNumCampos`.
 - Asigna los punteros dinámicos leídos. Inicializa `pacCitas` y `pacCancelaciones` explícitamente en `nullptr`.
2. `bool leerDoctor(ifstream& archivo, void*& doctor)`
 - Lee: DNI (int), Nombre (cadena), Especialidad (int).
 - Reserva un arreglo genérico de tamaño `docNumCampos` y guarda los datos.
3. `bool leerCita(ifstream& archivo, void*& cita, int& dniPaciente)`
 - Lee: Año, Mes, Día (ignora los guiones/comas y fusiona matemáticamente en un solo entero fecha con formato AAAAMMDD).
 - Lee: Hora, Minuto (fusiona en un solo entero horaCompleta con formato HHMM).
 - Lee: DNI del Paciente (guárdalo en la variable por referencia `dniPaciente`), DNI del Doctor, Estado.
 - Reserva tamaño `citaNumCampos` y guarda los datos (omite guardar el DNI del paciente en la cita).
4. `bool leerCancelacion(ifstream& archivo, void*& cancelacion, int& dniPaciente)`
 - Misma lógica de fecha y hora que la cita. Lee DNI Paciente por referencia. Lee DNI Doctor. Reserva tamaño `canNumCampos`.

FASE 2: Máquinas Genéricas de Carga

1. void cargarRegistros(const char* nombreArchivo, void*& datos, bool (*leerRegistro)(ifstream&, void*&))
 - Abre el archivo ios::in.
 - Implementa un bucle while (leerRegistro(archivo, dato)).
 - **Regla de Memoria (Incrementos):** Inicia con num = 0 y capacidad = 0. Si num == capacidad, invoca a una función auxiliar incrementarCapacidad que suma 5 a la capacidad, cree un nuevo arreglo void de tamaño capacidad + 1 (inicializado en nulo), copie los punteros existentes, libere el arreglo antiguo usando delete[] y asigne el nuevo.
2. void cargarEventos(const char* nombreArchivo, void*& clinica, int pos, bool (*leerEvento)(ifstream&, void*&, int&), void (*liberarEventoNoAsociado)(void*&))
 - Extrae el arreglo de pacientes de clinica (índice 0).
 - Bucle while que invoca a leerEvento.
 - Busca al paciente por el DNI retornado.
 - **Si existe:** Inyecta el evento en la posición pos (4 o 5) usando **Asignación Exacta** (crea un arreglo de tamaño cantidad_actual + 2, copia los viejos, inserta el nuevo, pon nullptr al final, haz delete[] al arreglo viejo).
 - **Si NO existe:** Invoca explícitamente a liberarEventoNoAsociado(dato).

FASE 3: Ensamble y Procesamiento

1. void cargarDatos(void*& clinica, const char* archPac, const char* archDoc, const char* archCit, const char* archCan)
 - Crea el arreglo raíz clinica con tamaño cliNumCampos.
 - Llama a cargarRegistros dos veces (para pacientes y doctores) pasándole los nombres de archivo y los callbacks leerPaciente y leerDoctor.
 - Llama a cargarEventos dos veces (citas y cancelaciones) pasando los callbacks de lectura y liberación correspondientes.
2. void cancelarCitas(void* clinica)
 - Recorre iterativamente el arreglo de pacientes buscando en su índice pacCancelaciones.

- Si encuentra cancelaciones, extrae el DNI del doctor, la fecha y la hora. Busca en el arreglo pacCitas una coincidencia exacta.
- Si hay coincidencia, cambia el estado de esa cita (índice 3) a 0.

FASE 4: Destrucción de Memoria

1. void liberarClinica(void*& clinica)

- Debe invocar a liberarPacientes y liberarDoctores.
- Cada paciente debe liberar sus arreglos internos de citas y cancelaciones, luego sus datos básicos (delete nombre, delete dni, etc.), y finalmente delete[] paciente.
- El programa debe compilar sin advertencias y, de ejecutarse con herramientas de análisis de memoria, reportar **0 bytes filtrados**.